Embedded Systems - Implementing filters on a Lattice FPGA

Gianpaolo Bianchi Ilenia Vinciguerra

Academic year 2025/2026

Contents

1 Introduction **3**

2 FPGA specification **3**

3 Filters **5**
- 3.1 Combinatorial filters 5
  - 3.1.1 Grayscale 5
  - 3.1.2 RGB to YCbCr 6
  - 3.1.3 Solarize 7
- 3.2 Kernel filters 7
  - 3.2.1 Sobel 7
  - 3.2.2 Emboss 9

4 Implementation **10**
- 4.1 Implementing the filters 10
  - 4.1.1 Combinatorial filters 10
  - 4.1.2 Kernel filters 11
- 4.2 Implementing the switching logic 11
  - 4.2.1 RGB switching logic 11
  - 4.2.2 "Brightness" switching logic 12

5 Timing and performance **12**
- 5.1 Comparison with the original demo 13

6 Practical guide to programming the FPGA **14**

7 Further specifications for future expansion **15**

8 Conclusions **16**

1 Introduction

This project consisted in implementing some basic filters on a Lattice Embedded Vision Development Kit (EVDK): a unique kit composed of three boards equipped with a dual CSI-2 camera outputting to High Definition Multimedia Interface (HDMI). Since the kit does not provide any external input device, the goal was also designing and implementing a switching logic which uses camera inputs to allow users to switch between the filters that we implemented.
The end goal for this project was achieving an aesthetically interesting and captivating result that could be used for divulgation purposes (Open Days at Politecnico di Milano), but which could also stimulate us to dive deeper in FPGA programming, starting from a practical and visual approach.
In this report, we will focus on detailing the implementation approach that we followed, starting from the initial familiarization phase with the particular hardware all the way down to the final working demo. We will also briefly cover the mathematical codification of the filters, which was the initial point of our implementation process. Finally, we will give an overview of the experimental results concerning timing and performance.

2 FPGA specification

The initial step of our process was gaining a basic understanding of the architecture of the EVDK, using the official documentation provided by Lattice to understand how and where we could actually implement our filters. In order to contextualize our work, we will give a brief overview on the result of our initial research. More detailed information about the kit can be retrieved in the official documentation available on the Lattice website.

The EVDK is composed of three boards which are physically stacked one on top of the other. Following the data stream, starting from the sensors all the way to the HDMI output, the first board in the processing chain is the CrossLink VIP Bridge Board, where the cameras are physically mounted. This board receives the MIPI video stream coming from the cameras, extracts the video pixels and merges the two streams. The result of the merge is then transmitted to the ECP5 FPGA which processes the merged sensor image and sends it to the HDMI VIP Output board that handles the final transmission via HDMI to the monitor.

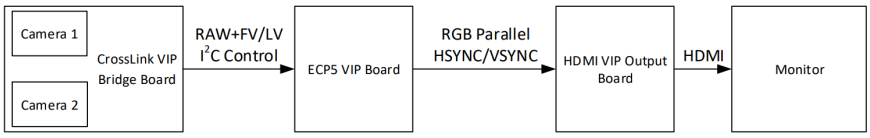


Figure 2.1. 2:1 MIPI CSI-2 to HDMI Bridge System Diagram

Lattice provides a ready to use demo that includes all the basic modules necessary to display the video captured by the cameras in a parallel view that looks like this on a monitor:

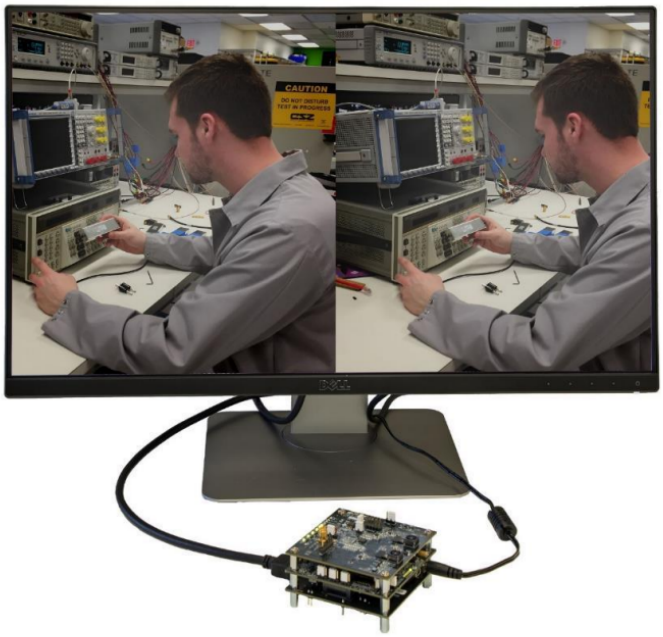


Figure 3.1. Dual Camera to HDMI Setup

Starting from the basic demo, we only had to work on the ECP5 FPGA, as the filters we implemented are all RGB based. Functionally, we added a new module between the pre-existing Color Space Converter and Gamma Correction modules for each filter and for the switching logic, leaving the rest of the demo intact.

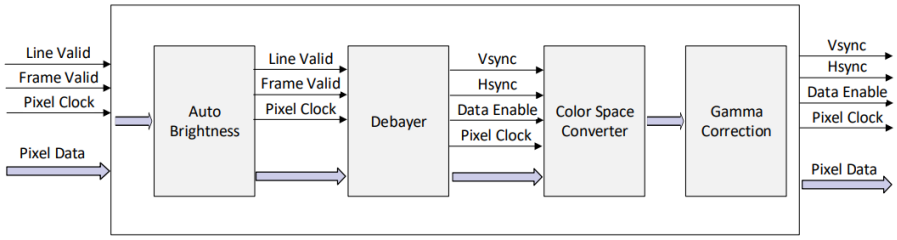


Figure 2.3. ECP5 Functional Block Diagram

To obtain a more visually appealing result, the filters are only applied to the right half of the screen. In order to achieve this while also keeping the code more readable and logically concise, we decided not to act on the CrossLink

which, in the demo, merges the two video streams in a single one. Acting on the CrossLink would have meant bypassing the merge. Instead, we went for an easier and more intuitive approach acting directly on the ECP5: we manually decide which filters have to be filtered, according to their position on the screen.

3 Filters

Our initial study process also consisted in researching some filters that are known in the literature and deciding which ones to implement, also depending on their mathematical codification.

The filters we decided to implement can be classified into two categories:

- **Combinatorial filters**: they operate pixel-by-pixel independently; therefore, they do not intrinsically require extra pipeline stages or memory and can be directly implemented as wires or basic arithmetic operations.
- **Kernel filters**: such as Sobel and emboss, which introduce some latency inside the image processing and mainly require line buffers.

3.1 Combinatorial filters

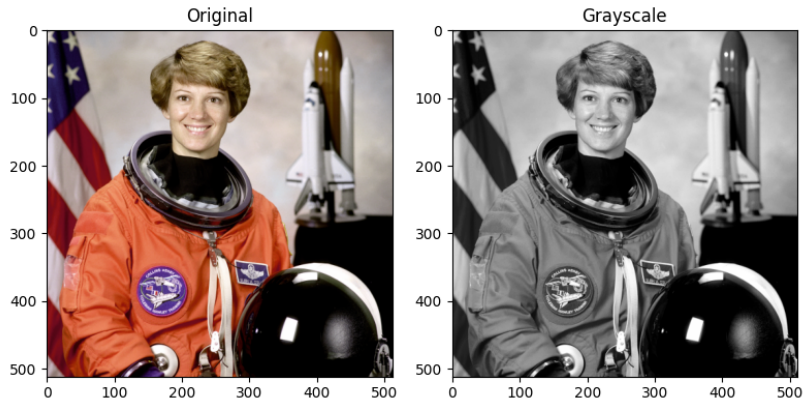
The combinatorial filters that we have selected are grayscale, RGB to YCbCr and solarize.

3.1.1 Grayscale

The application of a grayscale filter maps the original RGB values of the pixels in the original scene to values which codify colors ranging from black to white. The visual result is a frame in a range of grays where, numerically, R = G = B. Computationally, this is obtained by calculating a weighted average of the RGB color channels to obtain a value Y which will be used to set R = G = B = Y and to therefore obtain the gray. Following basic intuition, one could think that a simple average of the three channels would be enough to compute Y. However, the human eye does not perceive all colors equally: it is most sensitive to green, moderately sensitive to red, and least sensitive to blue. To account for this difference, several formulas that use different weights exist. We have chosen the following one, which is the right compromise between computational complexity and visual result:

$$Y = 0.3 * R + 0.59 * G + 0.11 * B$$

We can notice how green has the highest weight, exactly for the reason we have explained above.



3.1.2 RGB to YCbCr

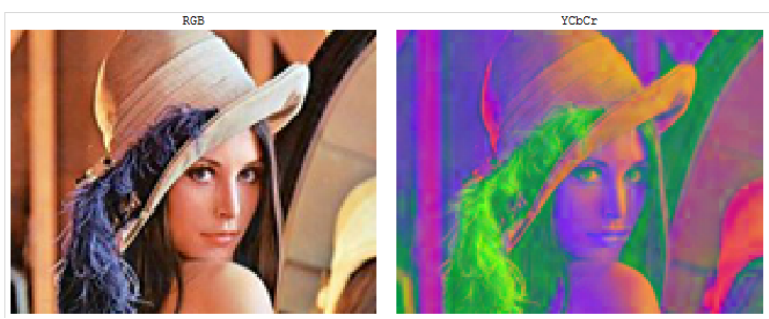
This filter converts images from their RGB encoding to YCbCr. The acronym represents the different channels that YCbCr uses, which are the following:

- Y (Luminance): the representation of the brightness of the light in the image
- Cb (Chroma Blue): the blue difference chrominance component (B - Y)
- Cr (Chroma Red): the red difference chrominance component (R - Y)

YCbCr is usually used for data compression, using the specific sensitivity of the human eye (way higher for brightness than for changes in color) as a leverage to encode images in a more memory efficient way without creating a noticeable change in their visual perception. In real applications, before images are visualized on a display they are reconverted back to RGB.

The standard formula to convert RGB into YCbCr is the following:

$$\begin{bmatrix} Y \\ C_b \\ C_r \end{bmatrix} = \begin{bmatrix} 0.299 & 0.587 & 0.114 \\ -0.168736 & -0.331264 & 0.5 \\ 0.5 & -0.418688 & -0.081312 \end{bmatrix} \begin{bmatrix} R \\ G \\ B \end{bmatrix} + \begin{bmatrix} 0 \\ 128 \\ 128 \end{bmatrix}$$



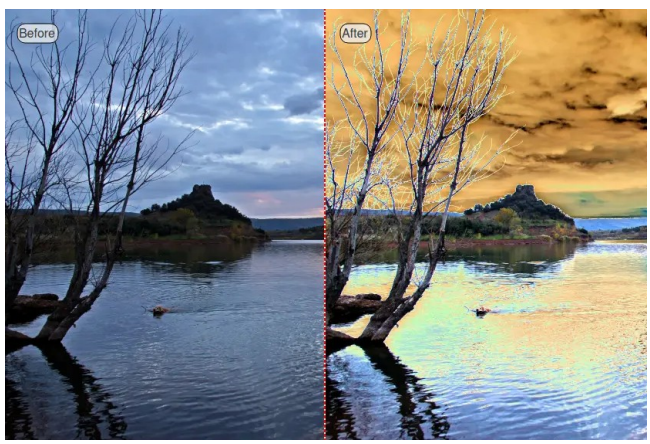
3.1.3 Solarize

Solarization (also known as the Sabattier effect) is a non-linear point transformation that re-maps pixel intensities based on a specific threshold.

For a given input channel $x \in R, G, B$ and a maximum possible value L, the transformation $f(x)$ is defined as:

$$f(x) = \begin{cases} x & \text{if } x \leq T \\ L - x & \text{if } x > T \end{cases}$$

where T is the set threshold and L is the so called bit-depth limit, which in this is case is set to 12 bits, meaning $L = 2^{12} - 1 = 4095$ levels. The bit-depth refers to the maximum number of unique values (shades) that a digital system can represent for a single color channel.



3.2 Kernel filters

As previously said, the kernel-filters we have chosen are Sobel and emboss. These filters work on a window (or kernel) of the pixels in the frame to perform operations, so they need information of neighboring pixels to be implemented, which in other words means having to use memory to store pixels.

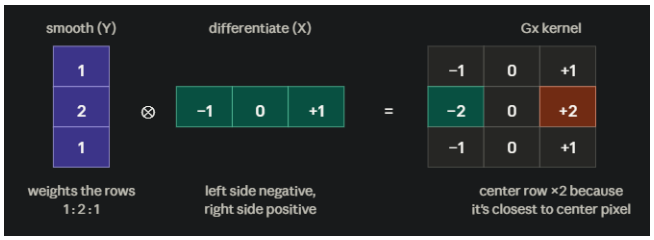
3.2.1 Sobel

The Sobel filter uses the so called Sobel operator which are normally used for edge detection in computer vision or image processing. In the case of a 3 x 3 Sobel filter, edges are detected using two 3 x 3 matrices to disclose the color changes in both the x and y directions.

$$G_x = \begin{bmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{bmatrix} \qquad G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ +1 & +2 & +1 \end{bmatrix}$$

Here we can notice how, for each pixel, applying the filter requires to use the $N-1$ surrounding rows of pixels.

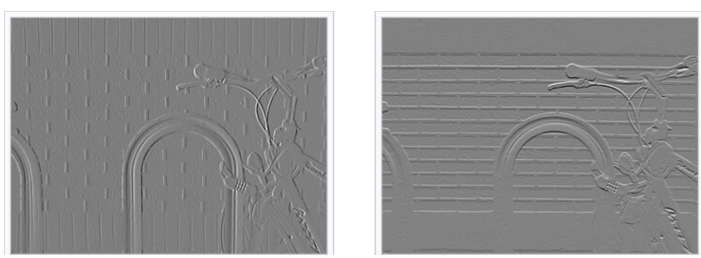
The matrices shown above are actually used to approximate derivatives in the x and y directions. The second matrix is the simplest approximation of a discrete derivative, whilst the first matrix uses binomial coefficients for smoothing: the weighting of the neighboring pixels.



The two matrices G_x and G_y each output an edge detection in the corresponding dimension. Finally, to know how much a pixel changes in any direction (given that both these matrices are "perpendicular"), we calculate the magnitude between them by computing:

$$G = \sqrt{G_x^2 + G_y^2}$$

The final output is a unique edge detection scenario.

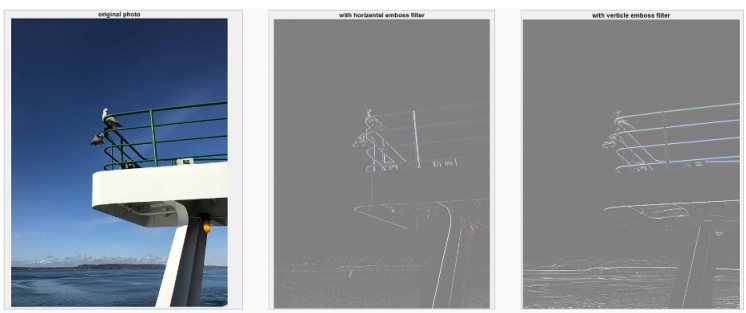


Sobel's matrix window can be arbitrarily enlarged (as long as it it remains smaller than the image frame) by following Pascal's triangle, but only using odd numbers (3 x 3, 5 x 5, ...), given that it needs to be a square matrix centered around one pixel.

The effect of increasing the window size is a higher robustness with respect to noise. This happens because of the increased smoothing effect but it comes at the expense of losing the ability to detect smaller edges. The sizing of the window therefore strictly depends on the specific case of application of the filter.

3.2.2 Emboss

The emboss filter follows a philosophy similar to the Sobel filter. The former is intentionally asymmetric and can be described by a number of matrices, all of which are directional difference filters, meaning they compute the edge in a particular direction whilst adding a bias and shifting the result to a visible gray midpoint.



Original image | Image (a): horizontal emboss | Image (b):vertical emboss

To create an emboss effect, the typical value that is computed is the difference in brightness (the derivative) between neighboring pixels in a specific direction. A variation of this filter is the weighted emboss filter, which adds a smoothing step to make the effect look less grainy. Finally, because the computation can result in negative numbers, a medium-gray "offset" is added so that flat, unchanging areas appear as neutral gray rather than solid black.

$$DiagonalEmb = \begin{bmatrix} -1 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & +1 \end{bmatrix} \quad SmoothingEmb = \begin{bmatrix} -2 & -1 & 0 \\ -1 & 0 & +1 \\ 0 & +1 & +2 \end{bmatrix}$$

Naming itself after the physical process of metal embossing, this filter generates a three-dimensional illusion. It achieves this by mapping pixel gradients as

highlights and shadows, effectively rendering a 'relief' of the original image atop a neutral gray field.

4 Implementation

For logistical and operational needs connected to the demo we have presented on the Open Days, we have worked in parallel on two different implementations of the same project which both follow the same rationale: implementing 3 filters, applying them only on the right side of the screen to obtain a side-to-side view of the filtered image with the original version, creating a switching logic which fully relies on the cameras to change the filter that is being currently applied. The specific characteristics of the two implementations are the following:

1. The first implementation only includes combinatorial filters (solely for divulgative reasons) and it uses a switching logic based on RGB color dominance, with specific thresholds that have been set empirically to trigger the switch only when a very saturated object is in the frame.

2. The second implementation only includes kernel filters and its switching logic is based on a brightness heuristic, specifically by counting the number of pixels whose grayscale value exceeds a fixed threshold (approximately 25% of the maximum intensity). If the number of such "bright" pixels falls below a predefined limit, indicating a generally dark scene, the active filter is switched after a cooldown period.

4.1 Implementing the filters

4.1.1 Combinatorial filters

The three combinatorial filters (grayscale, solarize and YCbCr) have been implemented in three different modules, all wired between the Color Space Converter and Gamma Correction modules. For each one of the filters, we have adopted specific approximations to lower the computational cost, which is critical since all the filters are pre-computed, independently from whether they are being applied or not.

1. **Grayscale**: the computation is performed as $Y = (77 * R + 150 * G + 29 * B) >> 8$ in a 2-stage pipeline: in the first stage only the multiplications are performed, while in the second stage the three results are added together, and the final result is shifted to the right.

2. **RGB to YCbCr**: the computation is a 3 x 3 matrix multiplication with the RGB vector and the sum of an offset. This is implemented using a 2-stages pipeline: in the first stage the coefficients are multiplied with the RGB vector, in the second stages the additions/differences are performed and the offset is added to the G and B channels.

3. **Solarize**: this filter is applied using the ternary operator in Verilog to set the result to $L - x$ if $x > T$ or to x otherwise. We have set the threshold $T = 1536$ to obtain a more accentuated effect.

4.1.2 Kernel filters

Given the small kernel-window size and the code complexity, the kernel filters were implemented in a single module, wired in the same place as the combinatorial filters. This allows us also to easily reuse some calculations such as the grayscale inside this second implementation.

Moreover every signal, independently of the active filter at that moment in time, is delayed by a number of L clocks, being L the maximum amount of latency introduced by a single filter. In this case, the emboss filter required fewer combinational operations, whilst Sobel, even though the small 3 x 3 kernel, introduced a lot of dependent arithmetic stages to be completed on a single clock. To keep up with the timing constraints provided by the FPGA, the datapath was broken into several registered pipeline stages. At 148.5 MHz, this cycles of latency do not affect perceived frame timing in a real-time display pipeline.

Given that pixels arrive as a serial stream in hardware, it is necessary to wait and save $N-1$ rows for a $N*N$ matrix before applying the filter for the centermost pixel, requiring memory and more pipeline stages, hence also introducing latency. A combination of horizontal shift registers and line buffers updates the 3 x 3 matrix at each pixel arrival to accommodate the new arriving values. This step forms the initial part of the pipeline responsible for constructing the 3×3 sliding window from the incoming pixel stream.

1. **Sobel**: multiple pipeline stages are introduced for the filter: calculate and buffer Sobel gradients; perform absolute value of kernels; Manhattan magnitude; final scale down and clamping to 12 bits of the output.

2. **Emboss**: the filter calculates the smoothing emboss kernel and, after the computation, the signed result is offset by +2048 to map it to the [0, 4095] range (mid-gray background), and then clamped to prevent overflow/underflow.

4.2 Implementing the switching logic

4.2.1 RGB switching logic

The first implementation of the switching logic triggers a filter switch whenever a new color becomes dominant in the frame. To achieve this, in a dedicated module the r_in, g_in and b_in signals of the pixels that belong to the left side of the screen (the unfiltered one) are accumulated for each frame. When a new frame arrives (vsync_in goes high) the RGB values obtained during the accumulation phase are stored in dedicated registers which are then compared between them to determine which channel is the dominant one. The comparison to establish the real dominant channel is performed by computing runner-ups

(the second "strongest" channel) and then checking if the distance between the dominant channel computed after the accumulation and the runner-up is more than a set margin. We calibrated the margins empirically to match the expected light conditions of the day of the demonstration but they can be easily adjusted. The previous dominance is stored in a register in order to trigger a switch only when a **NEW** color becomes dominant to avoid useless retriggers. When a new channel becomes dominant sel_latched is set to a specific two bits value, which is checked in the output mux to select the right filter to apply to the right part of the screen.

4.2.2 "Brightness" switching logic

In the second implementation, the switching logic is simplified. Instead of computing average luminance, a lightweight metric is used: the number of pixels whose grayscale value exceeds a fixed threshold (approximately 25% of the maximum intensity) is accumulated over each frame, providing then a low-cost approximation of scene brightness. If the number of such "bright" pixels falls below a predefined count threshold, indicating a predominantly dark scene, the active filter advances to the next one. A cooldown period of 3 seconds (180 frames at 60 fps) is also enforced to prevent rapid or unstable switching.

5 Timing and performance

In terms of memory, as previously said, combinatorial filters do not require any, since they rely on pure arithmetical computations.

In the case of the kernel-filters, with the specifics of a 3 x 3 window for both Sobel and emboss, we are introducing an initial latency of 2 frame lines and 4 clocks at steady-state.

It is worth noting that for larger kernel windows, the number of required pipeline stages could increase, as more arithmetic operations would need to be chained and potentially broken up to meet timing.

To avoid increasing code complexity, all signals are delayed the amount of clocks required for the longest filter (maximum amount of all filters), regardless if it is active at that time.

A minor visual artifact is present in the output: a one pixel border around the image where the 3×3 window cannot be fully formed, as the outermost pixels have no neighbors on one or more sides. These border pixels are computed using the initialized-to-zero values of the line buffers and shift registers, producing a faint rectangular enclosure visible at the edges of the filtered half of the frame. This issue could be easily fixed by simply checking if we are in the "border" of the filter, and compute some kind of partial filter or pass a predetermined output, but we chose to maintain simplicity and performance in the code given that it does not pose any problem.

For a 1920x1080 output image, there are two 12-bit line buffers each holding 1920 pixels are used to retain previous lines, which are the main cause for memory utilization as we can see in Table 1. There are also some registers to synchronize all signals in terms of the latency plus one 11-bit register to keep the column count.

Resource	**Used**	**Available**	**Usage**
SLICEs	4022	41820	10%
LUT4s	6734	83640	8%
Block RAMs (EBR)	59	208	28%
Registers	2422	84735	3%
DSP Multipliers	2	312	0.6%
PIO sites	64	365	18%

Table 1: Device resource utilization of the full project (kernel filters implementation)

The place-and-route completed successfully with no unrouted connections. The worst-case setup slack is 0.216 ns and the worst-case hold slack is 0.074 ns, both positive, confirming that all timing constraints are met with no violations. The total place-and-route runtime was 2 minutes and 42 seconds.

For the second implementation we have the following results:

Resource	**Used**	**Available**	**Usage**
SLICEs	1822	41820	4%
LUT4s	2763	83640	3%
Block RAMs (EBR)	41	208	20%
Registers	1899	84735	2%
DSP Multipliers	20	312	6%
PIO sites	64	365	18%

Table 2: Device resource utilization of the full project (combinatorial filter implementation)

The place-and-route completed successfully with no unrouted connections. The worst-case setup slack is 0.752 ns and the worst-case hold slack is 0.119 ns, both positive also in this case. The total place-and-route runtime was 1 minute and 44 seconds.

5.1 Comparison with the original demo

It is particularly significant to compare the previous experimental results with the ones of the original demo, in order to have an quantitative interpretation of

the impact of our implementation. We provide here the device resource utilization of the demo and the timing aspects:

Resource	**Used**	**Available**	**Usage**
SLICEs	1281	41820	3%
LUT4s	1948	83640	2%
Block RAMs (EBR)	41	208	20%
Registers	1288	84735	2%
DSP Multipliers	2	312	0.6%
PIO sites	64	365	18%

Table 3: Device resource utilization of the original demo

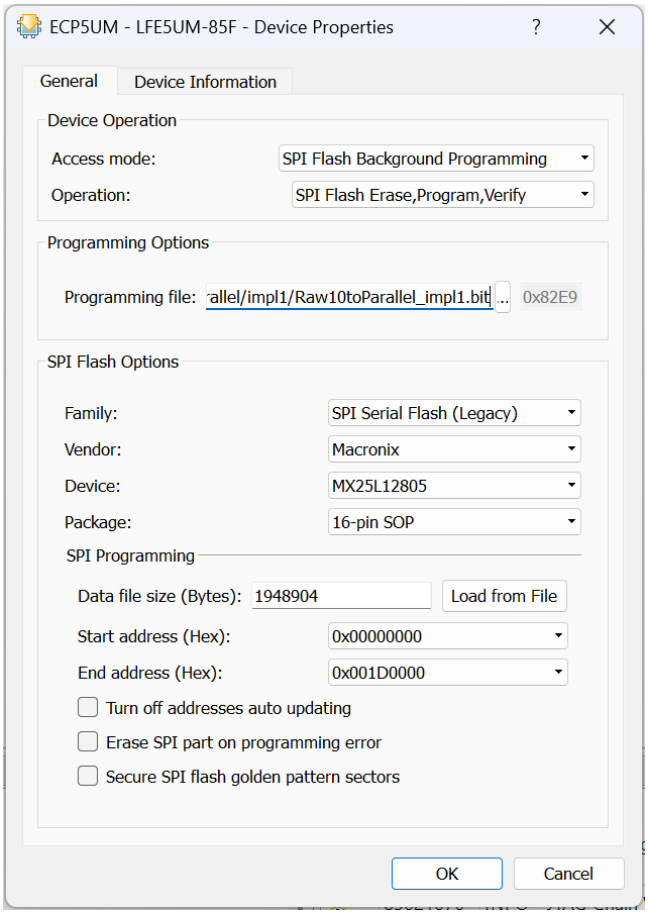
The place-and-route completes with no unrouted connections but with 90 potential circuit loops, which are of course also still present in our implementation. The worst-case setup slack is 0.299 ns and the worst-case hold slack is 0.113 ns, both positive, confirming that all timing constraints are met with no violations. The total place-and-route runtime was 1 minute and 48 seconds.

From this data we can conclude that our implementation, which introduces filters and switching logic, demonstrates a highly efficient use of FPGA resources. While logic utilization (SLICEs and LUT4s) increased significantly compared to the original demo, the overall footprint remains low at 10%, providing more than enough headroom for further scaling. The constant DSP usage in the kernel filters version is due to the fact that the filters were synthesized using distributed logic rather than dedicated hardware multipliers, which at this level is still an acceptable trade off, especially given the low resource density.
Timing wise, as logic utilization tripled, the timing margins decreased. The worst-case setup slack dropped by approximately 28% (from 0.216 ns to 0.299 ns) in the kernel filters implementation, while the hold slack also tightened. As expected, this does not fully apply to the combinatorial filters version, which uses less resources and has higher timing margins, due to the lower complexity of the filters.
Overall these reductions are expected in FPGA design when adding computational blocks like filters, as they increase routing density and logic levels. However, since all slack values remain positive, our implementation is still considered timing closed and reliable.

6 Practical guide to programming the FPGA

In order to run our demonstration, it is necessary to reprogram the ECP5 FPGA using the Lattice Programmer software. The first step is creating a blank project and adding a new device. The detailed instructions for these steps can be found in the official documentation provided by Lattice. Once the device has been

added correctly, using the SPI Flash Background Programming access mode and selecting the bitstream that we produced, it is possible to reprogram the kit and to then run the demonstration by simply powering the board and connecting it to a monitor via HDMI cable.



For the RGB switching logic, it is suggested to use three highly saturated objects or images to trigger the filter swap and, if necessary, to recalibrate the margins based on these objects. For the brightness logic, it is enough to cover the camera.

7 Further specifications for future expansion

During the open air demonstration of our project, we noticed that the kit's cameras are prone to saturate easily in bright natural light set ups, making the output images (both the filtered and unfiltered ones) very dark. In order to adapt to this, we suggest to completely remove the Gamma Correction module

which was implemented in the original demo and to correct min and max gain values in the auto-brightness module by lowering the values until satisfied.

8 Conclusions

This project allowed us to develop a working image processing pipeline on an FPGA platform, taking the Lattice EVDK from a basic camera demo to a multi filter video processing application. The result is a functional and visually engaging demonstration that was successfully presented at the Politecnico di Milano Open Days, achieving the original goal of combining technical depth with accessibility for a general audience. From a technical standpoint, the project gave us the opportunity to explore two fundamentally different categories of digital filters and to have to deal with tradeoffs regarding computational cost, memory requirements, and latency. The resource utilization data confirms that even with both filter families active, the design remains well within the capacity of the ECP5 FPGA, leaving significant headroom for future extensions. Beyond the specific filters implemented, this project served as a practical introduction to several key concepts in FPGA development: pipelining, line buffering, timing closure, and the integration of custom modules into a pre-existing datapath. Working with filters has given us the possibility to develop the project incrementally and to check the result at every intermediate step.
Finally, having to present the project to younger students has given us the chance to reflect deeper on how to effectively explain technical concepts in a way that is complete but easy to grasp and also to highlight the differences between working with software and hardware, which often not even university students have the possibility to explore directly.